Overview

The capstone consolidates the curriculum into an end-to-end system that can be demonstrated, evaluated, and explained in interviews. The goal is not maximum complexity; the goal is credible production-style design with measurable value.

Core integration target:

- LLM reasoning
- retrieval grounding
- agent orchestration
- monitoring and governance
- product-level success metrics

Capstone Project Options

Option 1: Enterprise Document Assistant (RAG + Agents)

Problem:

- Employees waste time searching policy and process docs.

System:

- retrieval agent finds evidence
- synthesis agent drafts answer with citations
- verifier agent checks citation grounding

Deliverables:

- grounded Q&A
- source attribution
- confidence and escalation path

Option 2: Agentic Customer Support Orchestrator

Problem:

- support teams manage repetitive multi-system workflows.

System:

- triage agent, policy retrieval agent, response agent, escalation agent

Deliverables:

- reduced time-to-first-response
- high-risk ticket routing
- action trace for audits

Option 3: AI Operations Assistant

Problem:

- incident response requires rapid triage and runbook execution.

System:

- anomaly summarizer, root-cause hypothesis agent, remediation recommender

Deliverables:

- alert prioritization

- probable root cause summaries
- safe action suggestions with approval gate

Capstone Architecture Template

Text diagram:

User/App -> API -> Orchestrator -> {Retriever, Planner, Worker Agents, Tool Gateway} -> Output + Trace Store

Required architectural elements:

1. Input validation and auth.
2. Context retrieval and prompt assembly.
3. Agent workflow (single or multi-agent).
4. Guardrails and policy checks.
5. Logging/tracing and error handling.
6. Evaluation harness.

Evaluation Criteria

Technical Robustness

- Task completion accuracy
- Groundedness/citation quality
- Latency and cost per task
- Failure recovery behavior

User and Product Quality

- User success rate
- subjective satisfaction (structured rubric)
- operational adoption metrics

Governance and Safety

- policy violation rate
- human override rate
- auditability completeness

Business Value

Use value framework:

$$\text{Net Impact} = \text{Time Saved} + \text{Risk Reduced} + \text{Revenue Lift} - \text{Run Cost}$$

Guidance & Pitfalls

Scope Management Rules

- Build one core workflow deeply before adding features.
- Start with deterministic tool interfaces.
- Define success metrics before coding.

Common Pitfalls

1. Overbuilding architecture before proving value.
2. Ignoring evaluation until demo week.
3. Missing observability, making failures impossible to diagnose.
4. No fallback path when retrieval/model confidence is low.

Recommended Build Sequence

1. Baseline pipeline (single path).
2. Add retrieval grounding.
3. Add agent loop and tool calls.
4. Add eval and monitoring hooks.
5. Add risk controls and human review.

Business Case Studies & Exceptions

Case Study 1: Enterprise Document Assistant Rollout

A consulting team built a capstone around policy-document Q&A with citation-grounded responses. The first demo looked strong, but pilot users reported low trust when responses lacked explicit source snippets.

Improvements applied:

- added top-k citation blocks with direct source excerpts
- introduced confidence threshold routing to human reviewer
- tracked groundedness and citation precision as mandatory metrics

Result:

- better user trust and lower correction effort by reviewers

Case Study 2: Agentic Support Workflow for Mid-Market SaaS

Another capstone implemented ticket triage and draft-response generation across CRM and helpdesk tools.

What worked:

- clear workflow boundaries (triage + draft only, no account mutations)
- escalation policies for financial and legal topics
- cost dashboard with token and latency budgets

What failed initially:

- no retry budget on tool timeouts, causing stuck tasks

Fix:

- bounded retries + deterministic fallback templates

Exceptions

- In highly regulated domains, capstone scope may need to be advisory-only with zero autonomous actions.
- Some enterprise clients require self-hosted models and private vector infrastructure, which can change scope and timeline significantly.

Interview Questions & Answers

1. **Describe your capstone in one minute.**
   State problem, architecture, measurable outcomes, and risk controls.
2. **How did you choose the use case?**
   Based on high-frequency pain point, measurable baseline, and feasible data/tool access.
3. **How did you evaluate model quality?**
   Task success, groundedness, latency, and cost metrics.
4. **What failures did you observe?**
   Retrieval misses, ambiguous prompts, tool timeouts, and mitigation strategy.
5. **How did you handle monitoring?**
   Added structured traces, step-level metrics, and alert thresholds.
6. **How did you manage safety?**
   Policy checks, blocked actions, and human-in-the-loop for high-risk tasks.
7. **Why not fine-tune instead of RAG?**
   RAG gave faster update cycle and easier source grounding for dynamic knowledge.
8. **How did you control costs?**
   Caching, model routing, context budgeting, and bounded retries.
9. **What was your rollback plan?**
   Fallback to deterministic baseline workflow and disable risky actions.
10. **If you had two more months, what would you improve?**
    Better eval harness, richer monitoring, and broader scenario coverage.
11. **How did you split deterministic vs LLM logic?**
    Deterministic for policy/transactions, LLM for interpretation and synthesis.
12. **What did you learn about productionizing agents?**
    Observability and failure design matter as much as prompt quality.
13. **How did you justify ROI?**
    Compared baseline process time/error against post-deployment metrics.
14. **What made your system trustworthy?**
    Citations, explicit uncertainty handling, and review gates.
15. **What architecture trade-off did you choose?**
    Simpler orchestrator initially, added complexity only where metrics demanded it.

References

- NYU Stern Foundations of AI Agents: https://aiagents.stern.nyu.edu/
- DeepLearning.AI Agentic AI: https://www.deeplearning.ai/courses/agentic-ai
- Anthropic effective agents: https://resources.anthropic.com/building-effective-ai-agents
- Google Cloud GenAI architecture guides: https://docs.cloud.google.com/architecture/genai-overview?hl=en
- Google enterprise GenAI/MLOps blueprint: https://docs.cloud.google.com/architecture/blueprints/genai-mlops-blueprint
- IITM Pravartak Applied AI with Context Engineering: https://futurense.com/iitm-pravartak/advanced-engineering-program-in-applied-ai-ml-with-context-engineering